SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title

Design and Implementation of a Multi-Platform Inventory Management System with RESTful API Backend

---

1. System Overview

1.1 Introduction

Retail businesses face challenges in managing inventory, tracking stock movements, monitoring sales, and generating business reports. Many small and medium-sized enterprises still rely on manual record-keeping or disconnected systems, leading to inaccurate stock records, stock shortages, overstocking, and poor decision-making.

This project proposes a Multi-Platform Inventory Management System that provides centralized inventory control through a RESTful API backend. The system will be accessible through web and desktop applications, allowing users to manage products, inventory, sales, suppliers, and reports from a unified platform.

1.2 Problem Statement

Many retail businesses experience the following problems:

- Manual inventory tracking is time-consuming and error-prone.
- Lack of real-time stock visibility.
- Difficulty tracking stock movement across branches.
- Inaccurate sales and inventory reports.
- Poor stock control leading to stock-outs and overstocking.
- Lack of centralized access to inventory information.
- Limited ability to monitor business performance.

These challenges reduce operational efficiency and negatively affect profitability.

The proposed system aims to address these issues by providing a centralized inventory management platform that improves inventory accuracy, reporting, and decision-making.

1.3 Project Goals

The system aims to:

- Automate inventory management processes.
- Improve stock accuracy and visibility.
- Simplify sales tracking and reporting.
- Support multi-branch inventory management.
- Provide centralized access to business data.
- Improve operational efficiency.

---

2. Stakeholder Analysis

Stakeholder	Role	Interests
Business Owner	Primary decision maker	Business performance, reports, inventory visibility
Administrator	System manager	User management, configuration, security
Branch Manager	Branch supervisor	Inventory monitoring, sales tracking
Cashier	Sales operator	Fast sales processing, receipt generation
Storekeeper	Inventory custodian	Stock-in, stock-out, inventory updates
Suppliers	Product suppliers	Purchase order processing
System Developer	System maintainer	Reliability, maintainability, scalability

---

3. Functional Requirements

User Management

FR-001: The system shall allow users to log in using valid credentials.

FR-002: The system shall allow users to log out securely.

FR-003: The system shall support role-based access control.

FR-004: Administrators shall create, update, and deactivate users.

Product Management

FR-005: The system shall allow authorized users to add products.

FR-006: The system shall allow authorized users to update product information.

FR-007: The system shall allow authorized users to deactivate products.

FR-008: The system shall allow users to search and view products.

FR-009: The system shall support barcode assignment for products.

Inventory Management

FR-010: The system shall record stock-in transactions.

FR-011: The system shall record stock-out transactions.

FR-012: The system shall maintain inventory history.

FR-013: The system shall track current inventory levels.

FR-014: The system shall generate low-stock alerts.

Sales Management

FR-015: The system shall allow cashiers to process sales.

FR-016: The system shall calculate transaction totals.

FR-017: The system shall generate receipts.

FR-018: The system shall store sales records.

FR-019: The system shall support multiple payment methods.

Supplier Management

FR-020: The system shall maintain supplier records.

FR-021: The system shall support purchase order creation.

FR-022: The system shall maintain purchase history.

Branch Management

FR-023: The system shall support multiple business branches.

FR-024: The system shall track inventory by branch.

FR-025: The system shall track sales by branch.

Reporting

FR-026: The system shall generate sales reports.

FR-027: The system shall generate inventory reports.

FR-028: The system shall generate stock movement reports.

FR-029: The system shall generate branch performance reports.

Desktop Application

FR-030: The desktop application shall support barcode scanning.

FR-031: The desktop application shall support receipt printing.

FR-032: The desktop application shall communicate with the REST API backend.

Notification Management

FR-033: The system shall notify users when stock reaches reorder level.

FR-034: The system shall notify administrators of failed synchronization operations.

FR-035: The system shall notify managers of branch inventory shortages.

Inventory Transfer Between Branches

FR-036: The system shall allow inventory transfer between branches.

FR-037: The system shall maintain inventory transfer history.

FR-038: The system shall approve or reject transfer requests.

Offline Synchronization

FR-039: The desktop application shall store transactions locally during network outages.

FR-040: The desktop application shall synchronize offline transactions with the central server when connectivity is restored.

FR-041: The system shall prevent duplicate synchronization of transactions.

---

4. Non-Functional Requirements

Performance

NFR-001: API responses shall be returned within 3 seconds under normal operating conditions.

NFR-002: The system shall support at least 100 concurrent users.

NFR-003: Inventory updates shall be reflected in real time.

Security

NFR-004: User passwords shall be encrypted before storage.

NFR-005: Authentication shall be implemented using JWT.

NFR-006: Access shall be controlled through user roles and permissions.

NFR-007: User sessions shall expire after a configurable inactivity period.

Reliability

NFR-008: System availability shall be at least 99%.

NFR-009: The Desktop app shall use offline SQLite if database is down and sync when database is run.

Scalability

NFR-010: The system shall support additional branches without significant performance degradation.

NFR-011: The architecture shall support future mobile application integration.

Usability

NFR-012: Users shall be able to complete common inventory operations within three clicks where practical.

NFR-013: The user interface shall be consistent across modules.

Maintainability

NFR-014: The system shall follow modular architecture.

NFR-015: API endpoints shall be documented using OpenAPI/Swagger.

Backup and Recovery

NFR-016: The system shall perform automatic database backups daily.

NFR-017: The system shall support restoration from backup within 30 minutes.

Audit Trail

NFR-018: The system shall record critical user activities in an audit log.

NFR-019: Audit logs shall be retained for at least one year.

Compatibility

NFR-020: The web application shall support modern browsers including Chrome, Edge, Firefox, and Safari.

NFR-021: Audit logs shall be retained for at least one year.

---

5. Context Diagram and System Boundary

Context Diagram (Textual Representation)

External Entities:

1. Administrator
2. Manager
3. Cashier
4. Storekeeper
5. Supplier System:

Multi-Platform Inventory Management

System Data Flows:

Administrator User Management, Reports

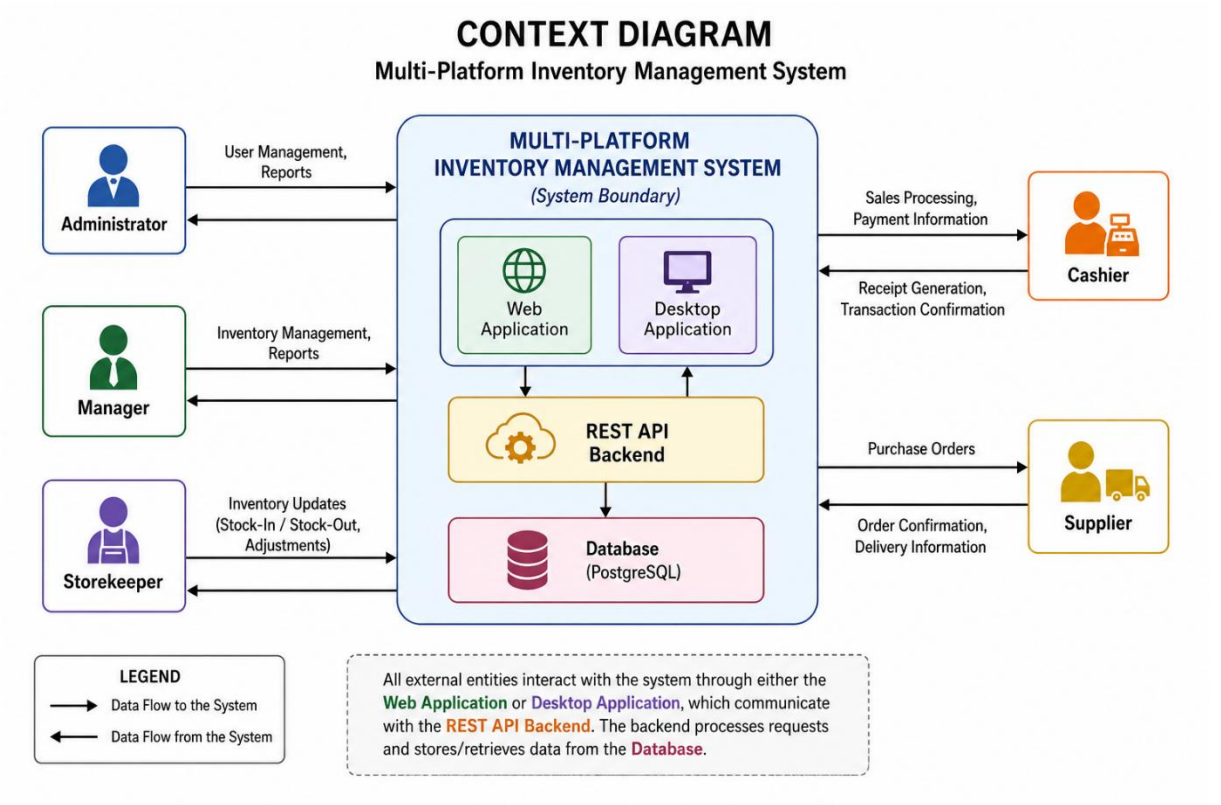
Manager Inventory Management, Reports

Cashier Sales Processing, Receipt Generation

Storekeeper Inventory Updates

Supplier Purchase Orders

All interactions occur through the Web Application or Desktop Application, which communicate with the REST API Backend and Database.



System Boundary

Inside the System Boundary:

- User Authentication
- Product Management
- Inventory Management
- Sales Management

- Supplier Management
- Purchase Management
- Reporting
- Notifications
- Barcode Processing

Outside the System Boundary:

- Physical Suppliers
- Customers
- Payment Providers
- Hardware Barcode Scanners
- Printers

---

6. Constraints

C-001: The project must be completed within the academic semester timeline.

C-002: PostgreSQL shall be used as the primary centralized database.

C-003: Node.js and Express.js shall be used for backend development.

C-004: React shall be used for web application development.

C-005: Electron shall be used for desktop application development.

C-006: Internet connectivity is required for real-time synchronization.

C-007: Available hardware resources may limit large-scale performance testing.

C-008: Budget limitations restrict deployment to cloud services with free or educational tiers.

---

7. Assumptions

A-001: Users possess basic computer literacy.

A-002: Retail businesses have internet connectivity.

A-003: Barcode scanners are compatible with desktop applications.

A-004: System users will receive basic training before use.

A-005: Business data entered the system is accurate.

---

8. Glossary

Term	Definition
API	Application Programming Interface
REST API	Architectural style for web services
Inventory	Goods available for sale
SKU	Stock Keeping Unit
POS	Point of Sale
Branch	Physical business location
Stock-In	Addition of inventory
Stock-Out	Reduction of inventory
Barcode	Machine-readable product identifier
Receipt	Proof of completed sale
UUID	Universally Unique Identifier
JWT	JSON Web Token used for authentication
Supplier	Entity that provides products to the business

---

9. Conclusion

The Multi-Platform Inventory Management System will provide a centralized platform for managing inventory, sales, suppliers, and business operations. The system is expected to improve inventory accuracy, enhance reporting capabilities, support multi-branch operations, and increase overall operational efficiency for retail businesses.

Link: https://github.com/benedictamankw18/Multi-Platform-Inventory-Management-System-with-RESTful-API-Backend